

Advanced Python

Inheritance

Day 6: Understand how classes reuse other classes. Stop repeating code.

Real-Life Example: Family

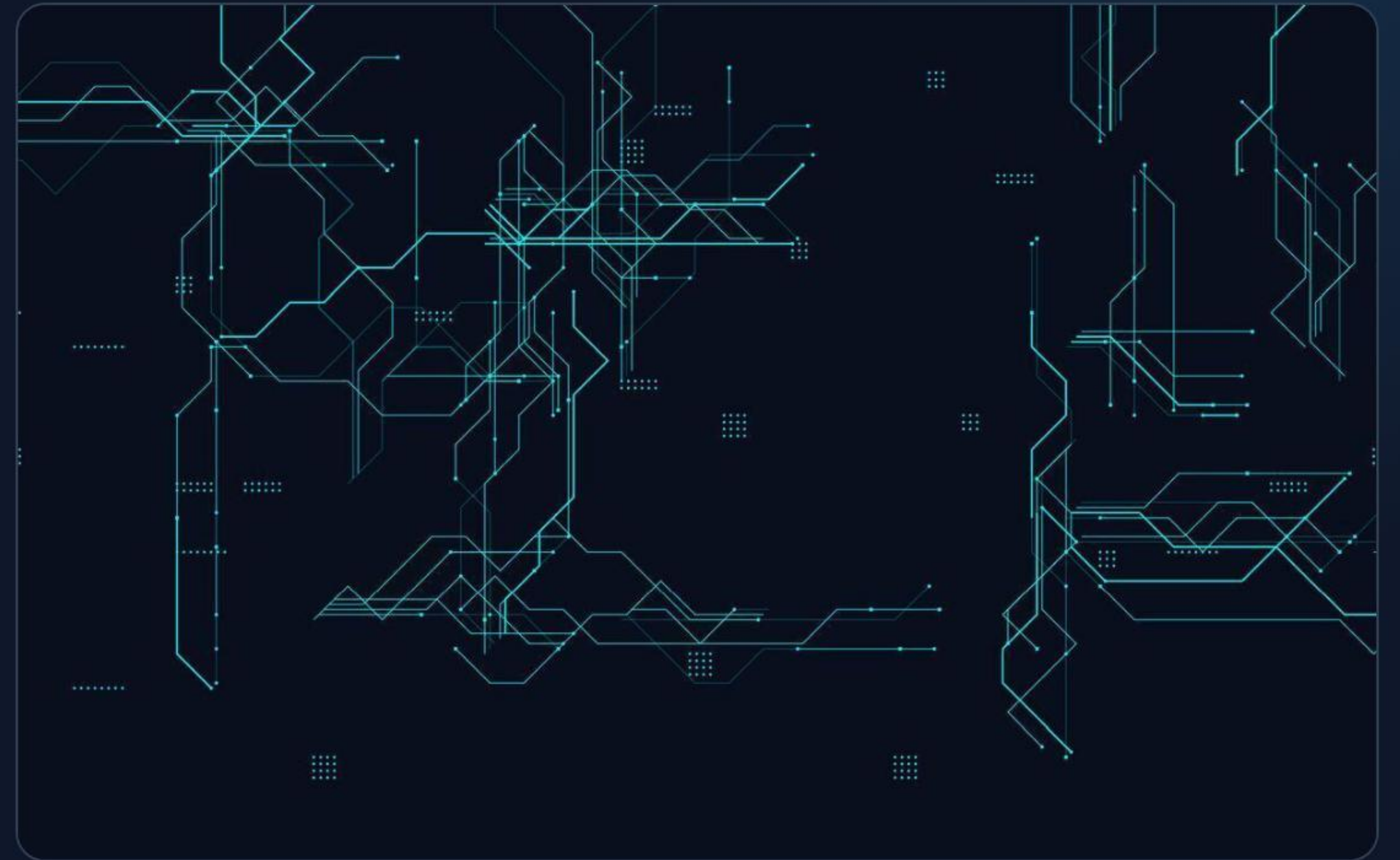
Father has:

- 🏠 House
- 📄 Surname
- 🚶 Habits

Child:

- ⬇️ Gets surname automatically
- 📄 May inherit habits
- ★ Can have own unique features

That is Inheritance in action.



What is Inheritance?



The Concept

Inheritance means a child class gets properties and methods from a parent class.

Instead of rewriting the same code, the child inherits the capabilities of the parent entirely.



Key Benefits

- ✓ **Code reuse:** Write once, use everywhere.
- ✓ **Clean structure:** Organized and logical hierarchy.
- ✓ **Scalability:** Easy to expand modular systems.

Real Life to Python Mapping

Real Life Concept	Python Equivalent	Description
Father / Parent	Parent Class	The base blueprint providing core traits.
Child	Child Class	The entity that receives the traits.
Inherited Property	Reused Code	Attributes and methods passed down.
New Skills	Extra Methods	Unique features added specifically to the child.

Basic Parent Class

What is a Parent Class?

A **Parent Class** (or Base Class) is the foundation of inheritance.

It defines the generic attributes and behaviors that can be shared among multiple specialized classes, establishing a blueprint that avoids repetitive coding.

Example Code

```
class Animal: def speak(self): print("Animal makes sound")
```

Here, `Animal` acts as the parent. It provides a generic `speak()` capability to any child that inherits it.

Basic Child Class

What is a Child Class?

A **Child Class** (or Derived Class) inherits from a parent by passing the parent's name in parentheses.

It automatically gains access to all the parent's methods and attributes without needing to redefine them.

Example Code

```
class Dog(Animal): pass
```

Dog inherits Animal. The pass keyword indicates we are not adding new features yet, but it possesses all Animal features.

Using Inheritance

Instantiation & Output

When you instantiate `d = Dog()` and call `d.speak()`, the output immediately prints "Animal makes sound".

The Dog class didn't define `speak()` itself; it inherited the exact behavior from `Animal`. This demonstrates the core power of true code reuse in action.



Base Blueprint

The Employee class serves as the core template. It defines the basic attributes and capabilities that every employee in the system must have.

 name attribute

 salary
attribute

 show_details()
method

Code Implementation

```
class Employee: def __init__(self, name, salary): self.name = name self.salary = salary
```


Code Implementation

```
class Manager(Employee):  
    def approve_leave(self):  
        print("Leave Approved")  
m = Manager("Saurav", 50000)  
m.
```


Vehicle Example

Classes Definition

```
# Parent Class class Vehicle: def start(self): print("Vehicle started") # Child Class class Car(Vehicle):
```


Power of Inheritance



Without Inheritance

- ✗ Rewrite the same attributes
- ✗ Duplicate method logic
- ✗ Messy, hard-to-read code
- ✗ Difficult updates



With Inheritance

- ✓ Reuse common code safely
- ✓ Maintain a clean structure
- ✓ Easy to maintain
- ✓ Scale cleanly as systems grow

Method Overriding

A child class can change its parent's behavior to suit its specific needs. This powerful feature is called **Method Overriding**.

```
class Animal: def speak(self): print("Animal sound") class Dog(Animal): def speak(self): print("Bark")
```

When `Dog().speak()` is called, it outputs **"Bark"**. The child's method takes precedence.

Think Before Running

The Code

```
class Person: def greet(self): print("Hello") class Student(Person): pass s = Student() s.greet()
```


Task 1: School System



Parent: Person

 **Attributes:** name, age

 **Methods:** introduce()



Child: Student

 **New Attribute:** grade

 **New Method:** study()

Task 2: Company System



Parent: Employee

 name

 salary



Child: Developer

+ programming_language



Child: Designer

+ design_tool

Tasks 3 & 4: Domains

Animal Kingdom

Parent: Animal → eat()

Child 1: Lion → roar()

Child 2: Bird → fly()

Banking Roles

Parent: BankUser → name, check_balance()

Child: Admin → approve_account()

Developer Thinking

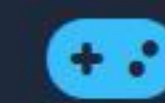


When to Use Inheritance?

- Many classes share common behavior
- You want a clean, logical hierarchy
- You want scalable, maintainable systems



Where is it Used?

-  Game characters (Enemy, Boss)
-  Employee systems (HR, Manager)
-  E-commerce roles (Buyer, Seller)
-  School systems (Student, Teacher)

Day 6 Summary

- ✓ Learned Inheritance concept
- ✓ Built Parent & Child classes
- ✓ Achieved ultimate Code Reuse
- ✓ Mastered Method Overriding

You now understand how large systems are structured.

Image Sources



https://static.vecteezy.com/system/resources/previews/067/827/001/non_2x/digital-circuit-structure-tech-lines-abstract-futuristic-background-for-technology-data-and-innovation-projects-minimalist-style-with-geometric-connections-clean-dark-design-illustration-vector.jpg

Source: www.vecteezy.com



<https://static.vecteezy.com/system/resources/thumbnails/055/369/907/small/abstract-representation-of-interconnected-nodes-and-lines-with-a-glowing-network-in-a-dark-setting-photo.jpg>

Source: www.vecteezy.com



https://img.freepik.com/premium-photo/digital-illustration-glowing-blue-gears-dark-circuit-pattern-background-conveying-concept-technology-innovation-3d-rendering_670147-110612.jpg?semt=ais_hybrid&w=740&q=80

Source: www.freepik.com



<https://static.vecteezy.com/system/resources/thumbnails/021/211/505/large/abstract-technology-motion-background-animation-with-a-flowing-blue-fractal-wave-of-glowing-blue-strings-of-light-shallow-depth-of-field-bokeh-full-hd-and-looping-video.jpg>

Source: www.vecteezy.com